CASE STUDY 2 CARVANA - ISBADBUY?

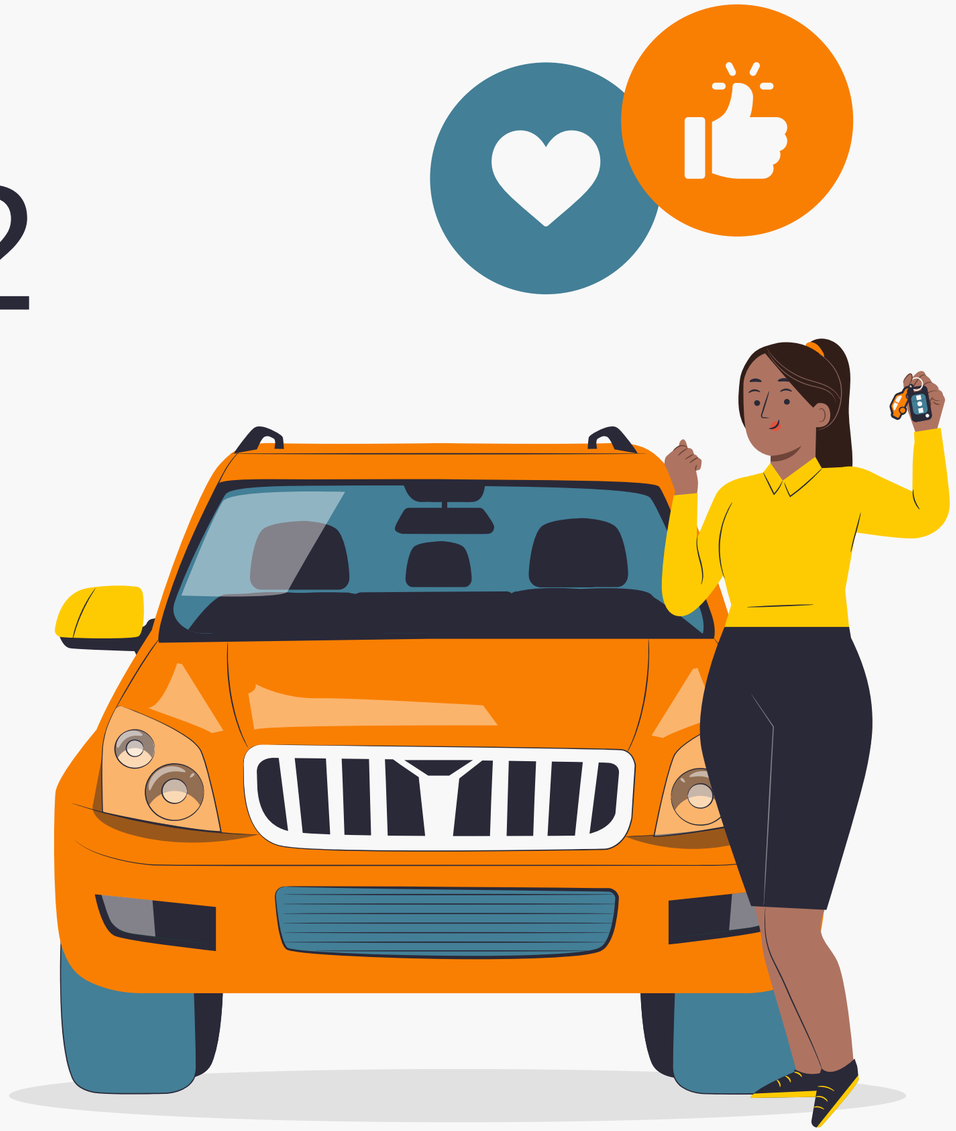


Table of contents

01
Case Introduction
Carvana background & IsBadBuy problem

02
Methodology
Variables, transforms & class imbalance

03
Coefficients
Logistic regression coefficient output

04
Training Results
Confusion matrix, thresholds & model fit, Hit Rate

05
Results Discussion
Expected vs. surprising findings

06
Conclusions
Kaggle results & key conclusions

01

Case Introduction

What Is Carvana? | The IsBadBuy Problem

Case Study: Carvana - IsBadBuy?

Using logistic regression to predict “bad buys” from auction data

Carvana sources a large share of its used car inventory through wholesale auto actions but purchases come with uncertainty. Vehicles can later reveal serious hidden issues and become costly “bad buys”. In this case, we use historical data to predict which vehicles are likely to become bad buys before purchasing using the Kaggle “Don’t Get Kicked” dataset

Why it matters:

- Hidden Issues are common at auctions: defects such as odometer problems, mechanical failures, and title issues may not be obvious at the purchase time but can create major losses.

- Bad buys: Losses aren’t just the vehicle itself but can be very costly after transportation, repair work, and market losses when reselling

- Inventory quality: This affects customer experience by having fewer bad buys, means more reliable vehicles and fewer returns/ complaints and better brand name over time

- Better decision improve profitability: Stronger screening improves margins while reducing operational waste such as rework, time, inventory, and resources tied up on problem vehicles

Background on Carvana

Online used-car retailer + auction based inventory

- Founded 2012 (Phoenix, AZ) is an online platform to research and purchase used cars (delivery or pickup)
- Vehicles are delivered or picked up (including “car vending machines” which are available in select markets)
- Rapid growth: by 2017 they entered approx. 60 metro markets and had roughly 10,000 vehicles in inventory
- A key capability of Carvana is their vertically integrated supply chain linked to the company’s proprietary software systems and data
- Uses proprietary internal/external data + algorithms to evaluate tens of thousands of vehicles daily (demand, profit, and inventory mix)
- Most of their inventory is acquired from wholesale auctions which creates a “bad buy” risk
- Because Carvana buys at scale, screening mistakes compound quickly which makes analytics operationally critical

Explain the Case

What is a “bad buy” and what data do we have?

- In auto auctions, some vehicles become costly “kicks” which are cars with serious hidden issues that prevent profitable resale

- Common issues include tampered odometers, mechanical problems, title issues, or other unexpected defects

- Carvana crowdsourced this prediction problem through Kaggle, an online data platform, where teams competed to predict bad buys to improve screening before purchase

<u>Data we are given</u>

- Training data: 72,983 auction purchases (2009-2010), labeled isBadBuy = 1 (is a bad buy) or 0 (not a bad buy)

- There is a class imbalance of about 12% of vehicles in training that are bad buys

- Test data includes 34 factors such as make, model, age, odometer, auction locations, etc.

- Teams are evaluated by how well predictions rank bad-buy risk on Kaggle

Objective for the case

Build a logistic regression model to predict isBadBuy risk

- Build: Train a logistic regression model using training dataset to predict isBadBuy (0/1)
- Explain: Use the data to justify variable selection chosen and interpret key drivers
- Report outcomes: Coefficients, standard errors, p-values
- Evaluate: Report pseudo-R , hit-rate , and confusion table on training data
- Apply: Score your training dataset, generate predicted probabilities, and submit to kaggle
- The goal is to produce a risk probability score to vehicles can be ranked from low risk to high risk before being purchased

02

Methodology

Variables, Feature Engineering
& Dataset Selection

Variable selection

For this case, we needed to predict whether a vehicle is classified as a BadBuy. For this model, we used a logistic regression with the following variables:

- **VehicleAge**
- **lnVehOdo (log of vehicle mileage)**
- **lnVehBCost (log of vehicle purchase cost)**
- **PRIMEUNT**
- **MMRCurrentAuctionCleanPrice**

Dependent variable: **IsBadBuy**

The next slides will show our process of choosing these variables.

Variable selection

Before we started our research, we had to review all the variables given in the case. Below is the data dictionary from Carvana:

Carvana: IsBadBuy?

Carvana Data Dictionary

	Field Name	Definition
1	RefID	Unique (sequential) number assigned to vehicles
2	IsBadBuy	Identifies if the kicked vehicle was an avoidable purchase
3	PurchDate	The date the vehicle was purchased at auction
4	Auction	Auction provider at which the vehicle was purchased
5	VehYear	The manufacturer's year
6	VehicleAge	The years elapsed since the manufacturer's year
7	Make	Vehicle manufacturer
8	Model	Vehicle model
9	Trim	Vehicle trim level
10	SubModel	Vehicle submodel
11	Color	Vehicle color
12	Transmission	Vehicle transmission type (manual or automatic)
13	WheelTypeID	The type ID of the vehicle's wheels
14	WheelType	The description of the vehicle's wheel type (alloy or covers)
15	VehOdo	The vehicle's odometer reading
16	Nationality	The manufacturer's country
17	Size	The size category of the vehicle (e.g., compact or SUV)
18	TopThreeAmericanName	Identifies whether the manufacturer is one of the top three American manufacturers
19	MMRAcquisitionAuctionAveragePrice	Acquisition price for the vehicle in average condition at time of purchase
20	MMRAcquisitionAuctionCleanPrice	Acquisition price for the vehicle in the above-average condition at time of purchase
21	MMRAcquisitionRetailAveragePrice	Acquisition price for the vehicle in the retail market in average condition at time of purchase
22	MMRAcquisitionRetailCleanPrice	Acquisition price for the vehicle in above-average condition at time of purchase
23	MMRCurrentAuctionAveragePrice	Acquisition price for the vehicle in average condition as of current day
24	MMRCurrentAuctionCleanPrice	Acquisition price for the vehicle in above-average condition as of current day
25	MMRCurrentRetailAveragePrice	Acquisition price for the vehicle in the retail market in average condition as of current day
26	MMRCurrentRetailCleanPrice	Acquisition price for the vehicle in the retail market in above-average condition as of current day
27	PRIMEUNIT	Identifies if the vehicle would have a higher demand than a standard purchase
28	AUCGUART	The level of guarantee provided by the auction for the vehicle
		(Green light = guaranteed/arbitratable; yellow light = caution/issue; red light = sold as is)
29	BYRNO	Unique number assigned to the buyer who purchased the vehicle
30	VNZIP	Zip code where the car was purchased
31	VNST	State where the car was purchased
32	VehBCost	Acquisition cost paid for the vehicle at time of purchase
33	IsOnlineSale	Identifies whether the vehicle was originally purchased online
34	WarrantyCost	Warranty price (term = 36 months and mileage = 36,000)

Variable selection

Next, we had to test each variable and see which variables were highly significant. To do this, we ran the coefficients, Std. errors, and the p-value of each variable. Below are the variables we have after testing:

	Field Name	Definition
1	RefID	Unique (sequential) number assigned to vehicles
2	IsBadBuy	Identifies if the kicked vehicle was an avoidable purchase
3	PurchDate	The date the vehicle was purchased at auction
4	Auction	Auction provider at which the vehicle was purchased
5	VehYear	The manufacturer's year
6	VehicleAge	The years elapsed since the manufacturer's year
7	Make	Vehicle manufacturer
8	Model	Vehicle model
9	Trim	Vehicle trim level
10	SubModel	Vehicle submodel
11	Color	Vehicle color
12	Transmission	Vehicle transmission type (manual or automatic)
13	WheelTypeID	The type ID of the vehicle's wheels
14	WheelType	The description of the vehicle's wheel type (alloy or covers)
15	VehOdo	The vehicle's odometer reading
16	Nationality	The manufacturer's country
17	Size	The size category of the vehicle (e.g., compact or SUV)
18	TopThreeAmericanName	Identifies whether the manufacturer is one of the top three American manufacturers
19	MMRAcquisitionAuctionAveragePrice	Acquisition price for the vehicle in average condition at time of purchase
20	MMRAcquisitionAuctionCleanPrice	Acquisition price for the vehicle in the above-average condition at time of purchase
21	MMRAcquisitionRetailAveragePrice	Acquisition price for the vehicle in the retail market in average condition at time of purchase
22	MMRAcquisitionRetailCleanPrice	Acquisition price for the vehicle in above-average condition at time of purchase
23	MMRCurrentAuctionAveragePrice	Acquisition price for the vehicle in average condition as of current day
24	MMRCurrentAuctionCleanPrice	Acquisition price for the vehicle in above-average condition as of current day
25	MMRCurrentRetailAveragePrice	Acquisition price for the vehicle in the retail market in average condition as of current day
26	MMRCurrentRetailCleanPrice	Acquisition price for the vehicle in the retail market in above-average condition as of current day
27	PRIMEUNIT	Identifies if the vehicle would have a higher demand than a standard purchase
28	AUCGUART	The level of guarantee provided by the auction for the vehicle
		(Green light = guaranteed/arbitratable; yellow light = caution/issue; red light = sold as is)
29	BYRNO	Unique number assigned to the buyer who purchased the vehicle
30	VNZIP	Zip code where the car was purchased
31	VNST	State where the car was purchased
32	VehBCost	Acquisition cost paid for the vehicle at time of purchase
33	IsOnlineSale	Identifies whether the vehicle was originally purchased online
34	WarrantyCost	Warranty price (term = 36 months and mileage = 36,000)

Featured Engineering Variables

To make our model more stable and better assess non-linear relationships, we transformed two variables:

1. VehOdo = **lnVehOdo**
2. VehBcost = **ln VehBCost**

The transformations that were done were log transformations for both the mileage and the purchase cost of each car.

lnVehOdo: logit$lnVehOdo ← log(logit$VehOdo + 1)

- Mileage is not a linear relationship with risk. The log transformation helps lower skewness and helps capture collapsing marginal effects.

lnVehBCost: logit$lnVehBCost ← log(logit$VehBCost + 1)

- Since the vehicle cost is right-skewed, the log transformation helps with variance and reduces the impact of outliers in the data.
- The log transformation helps with numerical stability and guides the logistic regression to model nonlinear relationships more impactfully.

Motivation for Variable Selection

In order to know which variables would be significant for our logistic regression, we tested each variables statisitcal significance and found that the following variables were the most statistically significant:

- **VehicleAge**
  - p-value: <2e-16 (Highly Significant)
  - Estimate = +0.2705
  - As vehicles get older, they are significantly more likely to be bad buys.
- **lnVehOdo**
  - p-value: 1e-15 (Highly Significant)
  - Estimate: = +0.4720
  - The higher the mileage, the higher the probability that the car will depreciate.
- **lnVehBCost**
  - p-value: <2e-16 (Highly Significant)
  - Estimate: -1.075
  - Higher-priced cars are less likely to be bad buys. Lower-priced cars are more likely to be a bad buy.
- **PRIMEUNIT**
  - PRIMEUNIT is a categorical variable, so two coefficients are present:
  - PRIMEUNITNULL = +1.576
  - PRIMEUNITYES = +1.478
  - This variable differs in significance in bad buy risk. This could suggest quality differences of vehicles across the inventory.
- **MMRCurrentAuctionCleanPrice**
  - p-value: 1.8e-15 (Highly Significant)
  - Estimate = 0.00006224
  - The acquisition price of the vehicle is above average for the current day. Market valuation at the auction has predictive power even if the per-dollar amount is small.

Output of Each Variable

Below shows the statistical significance of each variable. The output of these variables indicates high significance for our model

Variable	Estimate	Std.Error	Z-Value	P-Value
VehicleAge	0.2705	0.008489	31.86	<2e-16
lnVehOdo	0.472	0.05883	8.024	1.03e - 15
lnVehBCost	-1.075	0.06296	-17.083	<2e-16
PRIMEUNITYES	1.576	0.09231	17.077	<2e-16
PRIMEUNITNULL	1.478	0.4424	3.34	0.000839
MMRCurAuc	0.00006224	0.00000782	7.951	1.84e - 15

Column	Description
Estimate	Measures the log-odds of the variables being a BadBuy
Std. Error	Measures the variability of the estimate
Z Value	Test statistic = Estimate/Std. Error
P-value	measures the strength of evidence against null

Balanced or Imbalanced Dataset Selection Process

Balanced Dataset

logit	81959 obs. of 39 variables
logit_result	Large glm (30 elements, 72.6 MB)
null_result	Large glm (30 elements, 64.1 MB)
Values	
accuracy	0.634097536573165
hitrate	'table' int [1:2, 1:2] 46336 12318 17671 5634
pseudoR2	Class 'logLik' : 0.004657 (df=9)
sensitivity	0.313836898395722
specificity	0.723920821160186
threshold	0.23

Imbalanced Dataset

R Global Environment

Data

logit	72983 obs. of 38 variables
logit_result	Large glm (30 elements, 62.9 MB)
null_result	Large glm (30 elements, 57.1 MB)
Values	
accuracy	0.8541852212159
hitrate	'table' int [1:2, 1:2] 61046 7681 2961 1295
pseudoR2	Class 'logLik' : 0.05217 (df=7)
sensitivity	0.144273618538324
specificity	0.953739434749324
threshold	0.25

Observation: When we balanced our dataset, we noticed that our accuracy went from 85.4% to a 63.4%. This was an interesting observation since balancing the data would allow us to make a more accurate model, but it had the opposite effect.

Why did that happen?

- The accuracy decreased because the model doesn't just look at accuracy but class distribution as well. So when you add more bad buys, the model distribution shifts to become more "balanced." So when class proportion shifts, the accuracy will shift.

What we decided to do:

- Since logistic regression naturally handles imbalanced data, and our goal is to detect bad buys, we want our accuracy to be at its highest when detecting good buys over bad buys. After analyzing both the balanced and imbalanced data, we decided to use the imbalanced data to have the highest accuracy predicting good buys overall for our hit rate table.

03

Coefficients

Logistic Regression
Output

Coefficient R-Studio Output

```
Coefficients:
                            Estimate Std. Error z value Pr(>|z|)
(Intercept)                -1.020e+00  8.966e-01  -1.137 0.255457
VehicleAge                  2.705e-01  8.489e-03  31.860  < 2e-16 ***
lnVehOdo                    4.720e-01  5.883e-02   8.024 1.03e-15 ***
lnVehBCost                 -1.075e+00  6.296e-02 -17.083  < 2e-16 ***
PRIMEUNITNULL               1.576e+00  9.231e-02  17.077  < 2e-16 ***
PRIMEUNITYES                1.478e+00  4.424e-01   3.340 0.000839 ***
MMRCurrentAuctionCleanPrice 6.224e-05  7.827e-06   7.951 1.84e-15 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Positive coefficients increase bad-buy probability, while negative coefficients reduce it. VehicleAge, lnVehOdo, and PRIMEUNIT increase risk, while lnVehBCost reduces it.

Logistic Regression Coefficient Output

R glm( ) | family = binomial | 6 predictors

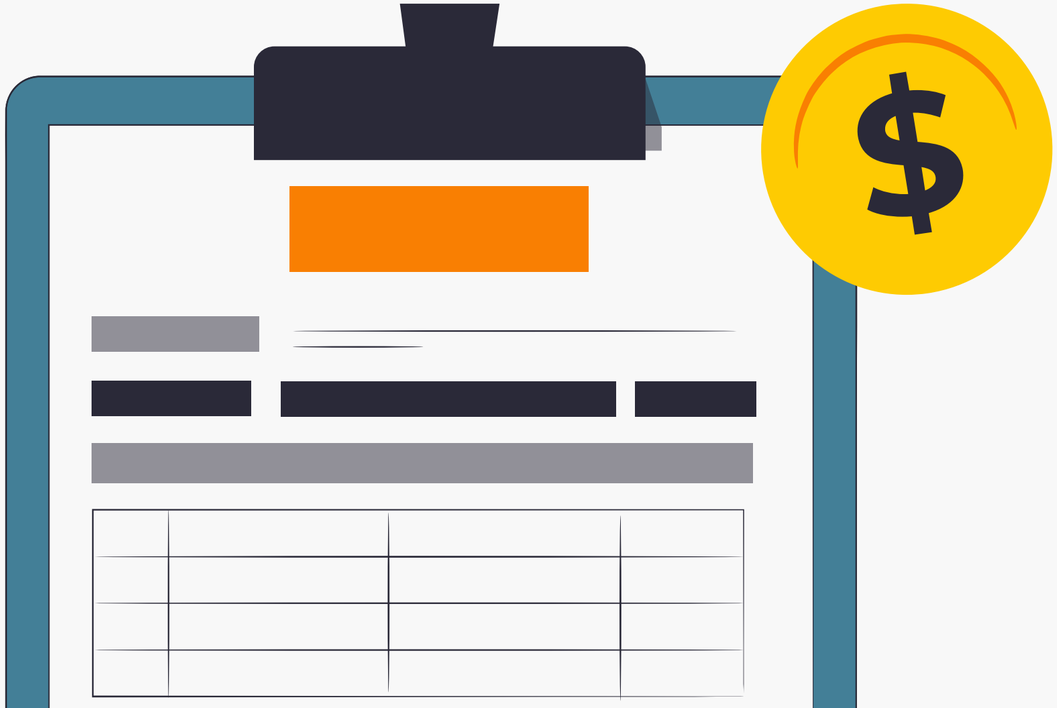
R Global Environment

Data	
logit	72983 obs. of 38 variables
logit_result	Large glm (30 elements, 62.9 MB)
null_result	Large glm (30 elements, 57.1 MB)
Values	
accuracy	0.8541852212159
hitrate	'table' int [1:2, 1:2] 61046 7681 2961 1295
pseudoR2	Class 'logLik' : 0.05217 (df=7)
sensitivity	0.144273618538324
specificity	0.953739434749324
threshold	0.25

Pseudo R^2 = 0.052 | AIC = 40,718 | Null deviance = 42,878 | Residual deviance = 40,658

04 Training Results

Confusion matrix, thresholds & model fit



Confusion Matrix and Probability Distribution

Threshold = 0.25 | Pseudo R^2 = 0.052 | Accuracy = 85.4%

```
Call:
glm(formula = IsBadBuy ~ VehicleAge + lnVehOdo + lnVehBCost +
    PRIMEUNIT + MMRCurrentAuctionCleanPrice, family = "binomial",
    data = logit)

Coefficients:
                            Estimate Std. Error z value Pr(>|z|)
(Intercept)                -1.020e+00  8.966e-01  -1.137 0.255457
VehicleAge                  2.705e-01  8.489e-03  31.860  < 2e-16 ***
lnVehOdo                    4.720e-01  5.883e-02   8.024 1.03e-15 ***
lnVehBCost                 -1.075e+00  6.296e-02 -17.083  < 2e-16 ***
PRIMEUNITNULL               1.576e+00  9.231e-02  17.077  < 2e-16 ***
PRIMEUNITYES                1.478e+00  4.424e-01   3.340 0.000839 ***
MMRCurrentAuctionCleanPrice 6.224e-05  7.827e-06   7.951 1.84e-15 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 54421  on 72982  degrees of freedom
Residual deviance: 51582  on 72976  degrees of freedom
AIC: 51596

Number of Fisher Scoring iterations: 5
```

Confusion Matrix @ 0.25

```
logit$pIsBadBuy <- ifelse(logit$predict >= threshold, 1, 0)
hitrate <- table(logit$IsBadBuy, logit$pIsBadBuy)
hitrate

      0     1
0 61046  2961
1  7681  1295
```

Predicted Probability Distribution

TN: 61,046 | FP: 2,961 | TP: 1,295 | FN: 7,681

- For our logistic regression model, we used a decision threshold of 0.25.
- The pseudo R^2 is 0.052, meaning the model explains about 5.2% of the variation in bad-buy probability.
- While that may sound small, this is typical in classification problems with noisy auction data.
- More importantly, model quality in classification is better evaluated using the confusion matrix and hit rate metrics, which we’ll walk through next.

Hit Rate by Decision Threshold

Impact of threshold choice on Accuracy, Sensitivity & Specificity

```
> null_result <- glm(IsBadBuy ~ 1, data = logit, family = "binomial")
> pseudoR2 <- 1 - logLik(logit_result)/logLik(null_result)
> pseudoR2
'log Lik.' 0.05217487 (df=7)
> logit$predict <- predict(logit_result, logit, type = "response")
>
> quantile(logit$predict, prob = seq(0,1,length=11), type=5)
        0%        10%        20%        30%        40%        50%        60%
0.00369238 0.05506713 0.07249323 0.08489982 0.09630158 0.10802368 0.12165754
       70%        80%        90%       100%
0.13859656 0.16437252 0.21186519 0.99951741
> threshold <- 0.25
```

Threshold	Accuracy	Sensitivity	Specificity	Notes
0.15	73.5%	43.9%	77.7%	High sensitivity, lower precision
0.20	82.0%	25.2%	89.9%	Balanced trade-off
0.25 ★	85.4%	14.4%	95.4%	Selected — high specificity

- Accuracy measures overall correct predictions.
- At 85.4%, the model appears strong, but this metric is misleading in imbalanced datasets.
- Sensitivity measures how well we detect actual bad buys.
- At 14.4%, we are only catching about 1 in 7 bad vehicles.
- Specificity measures how well we correctly identify good vehicles. At 95.4%, we are very strong at recognizing safe purchases.
- So the model prioritizes avoiding false alarms over catching risky vehicles.

Threshold Tradeoff Table (0.15, 0.20, 0.25)

Impact of threshold choice on Accuracy, Sensitivity & Specificity

Threshold	Accuracy	Sensitivity	Specificity	Notes
0.15	73.5%	43.9%	77.7%	High sensitivity, lower precision
0.20	82.0%	25.2%	89.9%	Balanced trade-off
0.25 ★	85.4%	14.4%	95.4%	Selected — high specificity

- As we increase the threshold from 0.15 to 0.25, overall accuracy increases from 73.5% to 85.4%, and specificity improves substantially.
- However, sensitivity drops sharply, meaning we miss more bad buys.
- There is no universally optimal threshold.
- The correct choice depends on the business cost of false negatives versus false positives.

Why accuracy increased?

- Good buys dominate the dataset.
- When we increase the threshold, we classify fewer vehicles as risky.
- This reduces false positives, and because most vehicles are good buys, accuracy increases.
- We tested balancing the dataset by oversampling bad buys.
- While it increased sensitivity in-sample, it significantly reduced Kaggle performance.
- Preserving real-world base rates produced better generalization.

“In summary, our model achieves high overall accuracy and strong specificity, but at the cost of low sensitivity.

The threshold decision ultimately depends on whether Carvana prioritizes avoiding false alarms or catching more bad inventory.”

Results
Discussion

05

Discussion

Expected Findings

- VehicleAge (+0.27): older cars → higher bad-buy risk
- lnVehOdo (+0.47): higher odometer → higher risk
- lnVehBCost (−1.075): expensive cars → lower risk
- Costly vehicles have stricter quality standards at auction
- PRIMEUNIT NULL (+1.576): unknown designation flags risk

Threshold Tradeoff

- At 0.25: only 14.4% sensitivity — 85.6% of bad buys missed
- Lowering to 0.15: sensitivity rises to 43.9%, but accuracy drops to 73.5%
- Accuracy improves as the threshold increases (73.5% → 85.4%)
- No single threshold is optimal — cost of each error type drives choice

Surprises

- MMRCleanPrice has tiny coef (+0.00006) yet is statistically significant
- High accuracy vs low sensitivity
- Pseudo R^2 = 0.052 — logistic regression captures only modest signal

06

Conclusions

Kaggle results & key conclusions

Why do we use testing data for our Kaggle submission?

The Test data simulates the real world

The test set contains cars the model has never seen before. Submitting predictions on this data is how we find out if our model can actually generalize - just like Carvana needs to flag bad buys on new cars arriving at auction every day, not ones it already evaluated.

It gives us an honest score

Because the model had no access to the test data during training, the Kaggle score reflects true predictive performance. This is why our training accuracy and Kaggle score differ slightly, and that difference is completely normal and expected.

How does Kaggle calculate the score?

Kaggle scores this competition using **Accuracy** - the percentage of vehicles classified as a bad buy or not.

The scoring formula:

Accuracy = Correct Predictions / Total Predictions

What counts as correct:

? **Predicted bad buy (1)**
Was actually a bad buy (1)

? **Predicted not bad buy (0)**
Was actually not bad buy (0)

? **Predicted bad buy (1)**
Was actually fine (0)

? **Predicted not bad buy (0)**
Was actually bad buy (1)

Kaggle Submission Results

Gini Coefficient (Normalized) = 2 × AUC – 1

kaggle _submission_3.csv Complete (after deadline) · now · VehicleAge+InVehOdo (created from VehOdo)+InVehBCost (create...	0.14634	0.14868	

- Private score: 0.14634 | Public score: 0.14868
- AUC ≈ (Gini + 1) / 2 = 0.573 — modest but better than random (AUC = 0.50)
- Gini = 1 → perfect discrimination | Gini = 0 → random guessing
- Result reflects parsimony: 6 predictors selected from 15 candidates
- Model traded maximum accuracy for interpretability and generalizability

What does our score mean?

85.4%	85.4%	14.4%	95.4%
Training Accuracy	Kaggle Accuracy	Sensitivity	Specificity

Interpreting the numbers

- Overall accuracy of 85.4% means our model correctly predicted whether a car was a bad buy or not for roughly 85 out of every 100 vehicles in the test.
- Sensitivity of 14.4% tells us that out of all the cars that were acutally bad buys, our model correctly flagged about 1 in7 of them.
- Specificity of 95.4% means that we almost never falsely accused a good car of being a bad buy - this protects Carvana from unnecessarily passing on good inventory.

Our findings

Key drivers of a bad buy

- **VehicleAge**: Older cars are significantly more likely to be bad buys - bad buys rates climb from 4% for 1-year-old cars to 31% for 9-year-old cars.
- **lnVehBCost**: Cheaper cars carry more risk - Carvana tends to apply less scrutiny to lower-cost purchases.
- **lnVehOdo**: Higher mileage increases the probability of being a bad buy.
- **MMRCurrentAuctionCleanPrice**: The market's benchmark price for the car is a significant predictor - large gaps between what was paid and what the market values the car are a warning sign.
- **PRIMEUNIT (NO)**: Cars flagged as NOT prime units are strongly associated with bad buys - non-prime cars have much less market demand, making them harder to asses accurately.

Surprising & notable results

- Cheaper cars are riskier (surprising): we expected higher-priced cars to be bad buys more often, but the data shows the opposite - lower-cost vehicles are more frequently misjudged at auction
- PRIMEUNIT is mostly missing: Over 95% of vehicles have no PRIMEUNIT value recorded. Despite this, the variable still carries significant predictive power for the cars where it is available.
- Brand matters less than expected: While Make showed variation in bad buys rates in the raw data, individual brands were mostly not statistically significant once other variables like age, mileage, and price were controlled for.
- The model is conservative: With a 0.25 threshold, our model catches only 14% of actual bad buys. This means most bad buys go undetected, but the ones flagged are flagged with high confidence.

Key takeaways from the Carvana case

1. Data preparation is everything
Most of our work was about cleaning the data, handling NULLS, applying log tranforms, and understanding the data so that we make informed decisions about the variables we keep.

2. Logistic regression is used when used correctly
Variable selection and thoughtful feature engineering matter more than model complexity.

3. Imbalanced data requires a specific threshold
Using a 0.5 threshold on a dataset where only 12% of outcomes are positive would classify almost everything as 0. Choosing an informed threshold is critical for meaningful predictions.

4. Real-worl accuracy looks different than expected
In business, understanding sensitivity and specificity matters as much as overall accuracy.

5. Market value is a key signal
The gap between what Carvana paid and what the market says a car is worth is one of the strongest signals of a bad buy.

6. Training vs. test performance
Evaluating a model only on training data gives an overly optimistic view. The Kaggle submission taught us why holdout data is essential for any honest evaluation of model quality.